

Rapport de TP : Bases de données NoSQL « MongoDB »

Module : Bases de Données NoSQL Réalisé par : Saleck BAYA - C25362 filiere : M1-SI

Introduction

L'objectif de ce TP est de se familiariser avec les bases de données orientées documents à travers le SGBD NoSQL MongoDB. L'environnement a été mis en place en démarrant un serveur mongod et un client interactif mongo. Nous avons ensuite manipulé une base locale avec des opérations CRUD, puis nous avons interrogé une base plus riche (gym) à travers des requêtes avancées et le paradigme Map-Reduce.

Exercice 1 : Installation, création d'une BD et opérations CRUD

1. Installation et lancement

Après avoir téléchargé et configuré les répertoires de données (C:\data\db), le serveur a été lancé, suivi du client interactif.

2. Serveur mongo

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\saleck\OneDrive\Bureau\BD_NoSQL\TP_BD-no-SQL>mkdir C:\data\db
C:\Users\saleck\OneDrive\Bureau\BD_NoSQL\TP_BD-no-SQL>cd C:\data\db
C:\data\db>mongod
{"t":{"$date":"2026-06-16T22:18:30.186+00:00"},"s":"I", "c":"-","id":8991280, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":"561833151"}}
{"t":{"$date":"2026-06-16T22:18:30.208+00:00"},"s":"I", "c":"CONTROL", "id":99374, "ctx":"thread1","msg":"Automatically disabling TLS 1.0 and TLS 1.1, to force-enable TLS 1.1 specify --sslDisabledProtocols 'TLS1_0'; to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
{"t":{"$date":"2026-06-16T22:18:30.319+00:00"},"s":"I", "c":"NETWORK", "id":4915701, "ctx":"thread1","msg":"Initialized wire specification","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":27},"incomingInternalClient":{"minWireVersion":0,"maxWireVersion":27},"outgoing":{"minWireVersion":6,"maxWireVersion":27},"isInternalClient":true}}}}
{"t":{"$date":"2026-06-16T22:18:30.320+00:00"},"s":"I", "c":"CONTROL", "id":5945603, "ctx":"thread1","msg":"Multi threading initialized"}
{"t":{"$date":"2026-06-16T22:18:30.321+00:00"},"s":"I", "c":"CONTROL", "id":4615611, "ctx":"initandlisten","msg":"MongoDB starting","attr":{"pid":6584,"port":27017,"dbPath":"C:/data/db","architecture":"64-bit","host":"Slk"}}
{"t":{"$date":"2026-06-16T22:18:30.321+00:00"},"s":"I", "c":"CONTROL", "id":23398, "ctx":"initandlisten","msg":"Target operating system minimum version","attr":{"targetMinOS":"Windows 7/Windows Server 2008 R2"}}
{"t":{"$date":"2026-06-16T22:18:30.321+00:00"},"s":"I", "c":"CONTROL", "id":23403, "ctx":"initandlisten","msg":"Build Info","attr":{"buildInfo":{"version":"8.2.5","gitVersion":"a471a13894434666c4a1f75451f2efa49f8f5df","modules":[],"allocator":"tcmalloc-gperf","environment":{"distmod":"windows"}}}}
{"t":{"$date":"2026-06-16T22:18:30.321+00:00"},"s":"I", "c":"CONTROL", "id":51765, "ctx":"initandlisten","msg":"Operating System","attr":{"os":{"name":"Microsoft Windows Workstation (build 26200)","version":"10.0 (build 26200)}}}}
{"t":{"$date":"2026-06-16T22:18:30.321+00:00"},"s":"I", "c":"CONTROL", "id":21951, "ctx":"initandlisten","msg":"Options set by command line","attr":{"options":{}}}
{"t":{"$date":"2026-06-16T22:18:30.322+00:00"},"s":"I", "c":"NETWORK", "id":11621101, "ctx":"initandlisten","msg":"Overriding max connections to honor 'capMemoryConsumptionForPreAuthBuffers' settings","attr":{"limit":201044}}
{"t":{"$date":"2026-06-16T22:18:30.323+00:00"},"s":"I", "c":"NETWORK", "id":4648601, "ctx":"initandlisten","msg":"Implicit TCP FastOpen unavailable. If TCP FastOpen is required, set at least one of the related parameters","attr":{"relatedParameters":{"tcpFastOpenServer","tcpFastOpenClient","tcpFastOpenQueueSize"}}}}
{"t":{"$date":"2026-06-16T22:18:30.329+00:00"},"s":"I", "c":"STORAGE", "id":22315, "ctx":"initandlisten","msg":"Opening WiredTiger","attr":{"config":{"create","cache_size=734M","session_max=33000","eviction=(threads_min=4,threads_max=4)","config_base=false","statistics=(fast)","log=(enabled=true,remove=true,path=journal,compressor=snappy)","builtin_extension_config=(zstd=(compression_level=6)),file_manager=(close_idle_time=600,close_scan_interval=10,close_handle_minimum=2000),statistics_log=(wait=0),json_output=(error_message)","verbose=(recovery_progress:1,checkpoint_progress:1,live_restore_progress:1,backup:0,checkpoint:0,compact:0,eviction:0,fileops:0,historical:0,live_restore:0,recovery:0,timestamp:0,tiered:0,transaction:0,verify:0,log:0),prefetch=(available=true,default=false)"},"}}
{"t":{"$date":"2026-06-16T22:18:30.339+00:00"},"s":"I", "c":"WIRETIGER", "id":22430, "ctx":"initandlisten","msg":"WiredTiger message","attr":{"message":{"ts_sec":1781648310,"ts_usec":339423,"thread":"6584:140723736970784","session_name":"wiredtiger_open","category":"WT_VERB_RECOVERY","log_id":1000000,"category_id":32,"verbose_level":"INFO","verbose_level_id":0,"msg":"Opening the WiredTiger library"}}}}
{"t":{"$date":"2026-06-16T22:18:30.358+00:00"},"s":"I", "c":"WIRETIGER", "id":22430, "ctx":"initandlisten","msg":"WiredTiger message","attr":{"message":{"ts_sec":1781648310,"ts_usec":358429,"thread":"6584:140723736970784","session_name":"connection","category":"WT_VERB_RECOVERY","log_id":1000000,"category_id":32,"verbose_level":"INFO","verbose_level_id":0,"msg":"connection configuration string parsing completed"}}}}
{"t":{"$date":"2026-06-16T22:18:30.396+00:00"},"s":"I", "c":"WIRETIGER", "id":22430, "ctx":"initandlisten","msg":"WiredTiger message","attr":{"message":{"ts_sec":1781648310,"ts_usec":395136,"thread":"6584:140723736970784","session_name":"connection","category":"WT_VERB_RECOVERY","log_id":1000000,"category_id":32,"verbose_level":"INFO","verbose_level_id":0,"msg":"Starting WiredTiger utility threads"}}}}
{"t":{"$date":"2026-06-16T22:18:30.419+00:00"},"s":"I", "c":"WIRETIGER", "id":22430, "ctx":"initandlisten","msg":"WiredTiger message","attr":{"message":{"ts_sec":1781648310,"ts_usec":418490,"thread":"6584:140723736970784","session_name":"connection","category":"WT_VERB_RECOVERY","log_id":1000000,"category_id":32,"verbose_level":"INFO","verbose_level_id":0,"msg":"Starting WiredTiger recovery"}}}}
{"t":{"$date":"2026-06-16T22:18:30.420+00:00"},"s":"I", "c":"WIRETIGER", "id":22430, "ctx":"initandlisten","msg":"WiredTiger message","attr":{"message":{"ts_sec":1781648310,"ts_usec":419605,"thread":"6584:140723736970784","session_name":"txn-recover","category":"WT_VERB_RECOVERY","log_id":1000000,"category_id":32,"verbose_level":"INFO",
```

Figure 1: mongod

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Microsoft Windows [version 10.0.26200.8655]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\salec>mongosh
Current Mongosh Log ID: 6a31cbe51561b4e9867c2906
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.2.5
Using Mongosh:       2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-06-11T16:28:33.258+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> |
```

Figure 2: mongosh

3. client mongo

5. Création d'une BD Mongo et opération CRUD

J'ai sélectionné la base `info` (ce qui la crée à la volée) et j'ai inséré trois documents dans la collection `produits`.

//question a

```
use info
```

//question b

```
db.produits.insert({nom: "Macbook Pro", fabricant: "Apple", prix: 11435.99, options: ["Intel Core i5",
```

//question c

```
db.produits.insert({nom: "Macbook Air", fabricant: "Apple", prix: 125794.73, ultrabook: true, options:
```

//question d

```
db.produits.insert({nom: "Thinkpad X230", fabricant: "Lenovo", prix: 114358.74, ultrabook: true, option
```

//question e //Afficher tous les produits

```
db.produits.find().pretty()
```

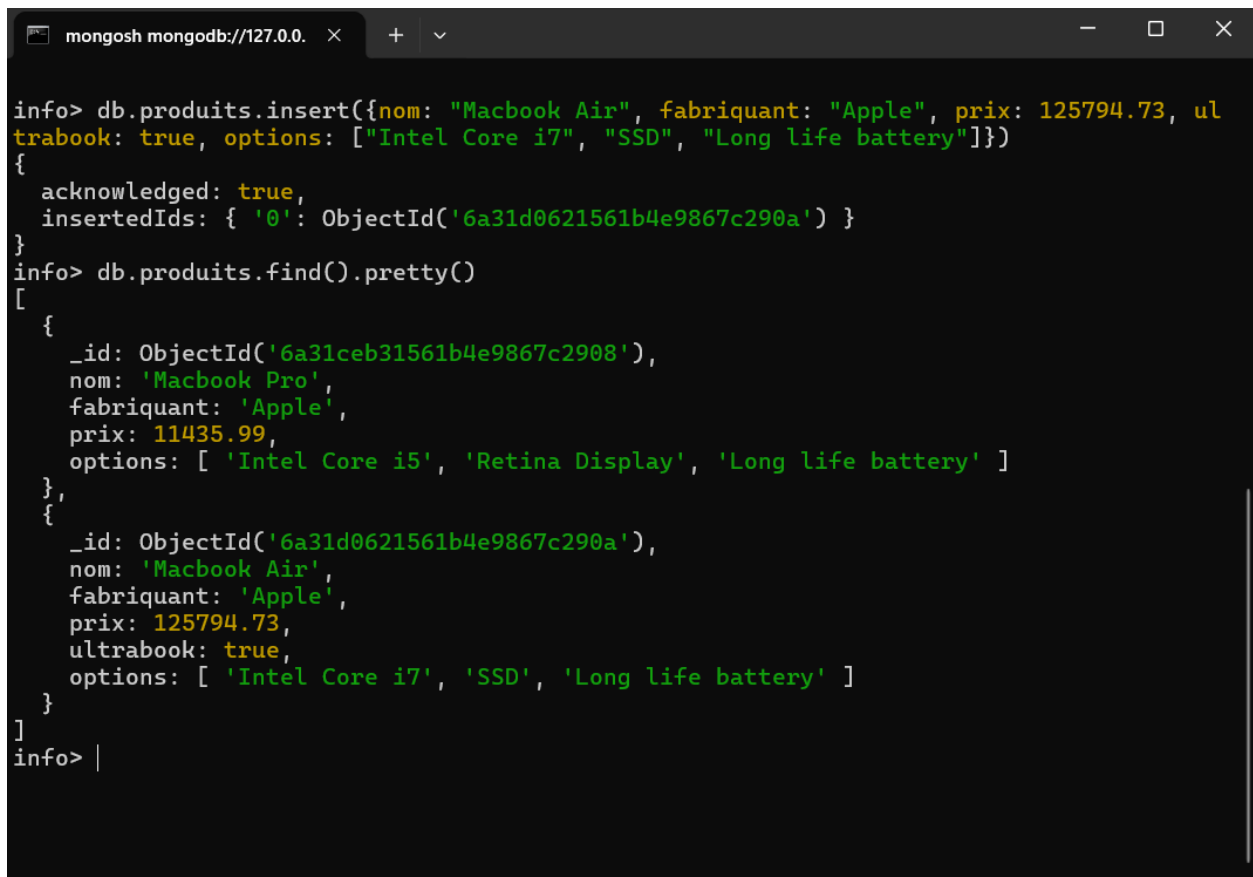
Récupérer le premier produit :

```
mongosh mongodb://127.0.0.1 x + v
Using MongoDB:      8.2.5
Using Mongosh:      2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----

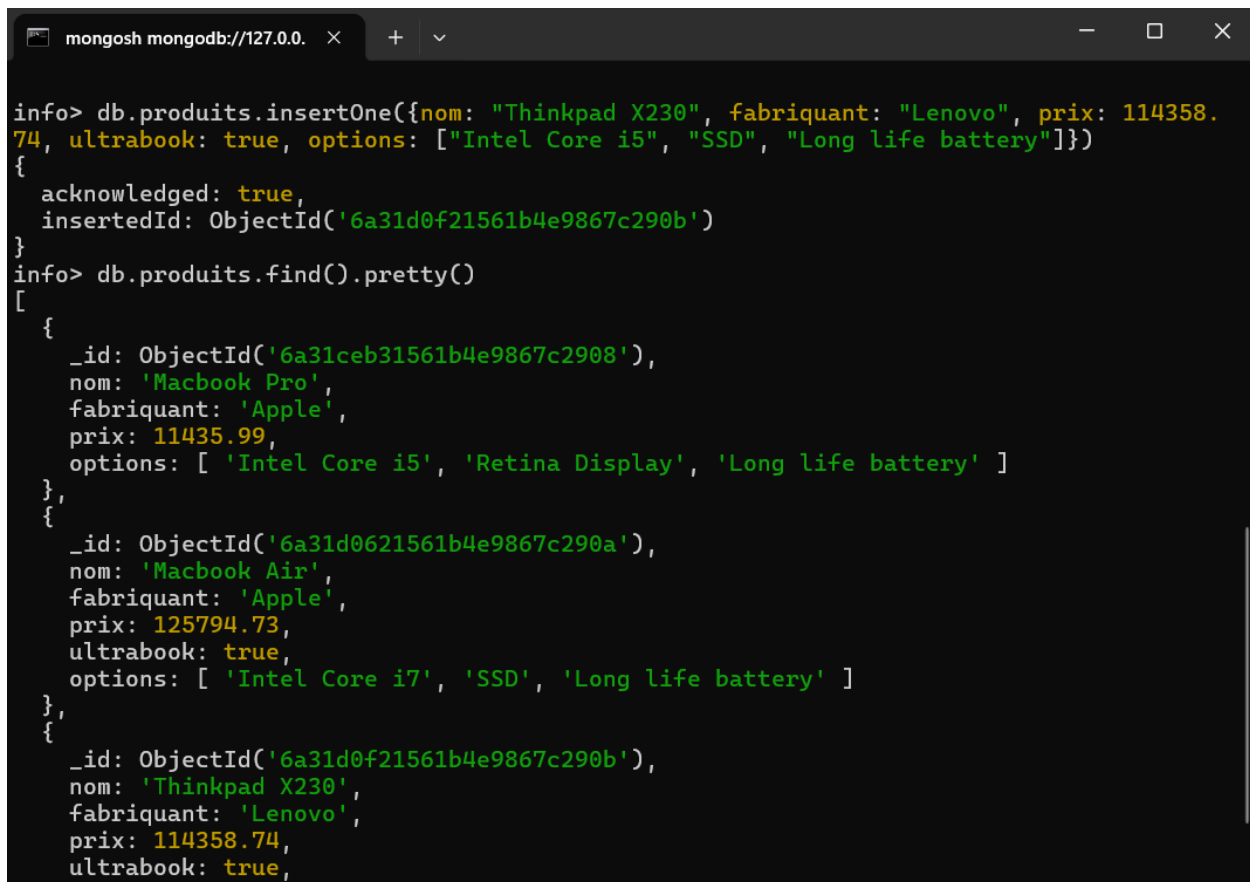
info> db.produits.insertOne({
|   nom: "Macbook Pro",
|   fabricant: "Apple",
|   prix: 11435.99,
|   options: ["Intel Core i5", "Retina Display", "Long life battery"]
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a31ceb31561b4e9867c2908')
}
info> show collections
produits
info> db.produits.find().pretty()
[
  {
    _id: ObjectId('6a31ceb31561b4e9867c2908'),
    nom: 'Macbook Pro',
    fabricant: 'Apple',
    prix: 11435.99,
    options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
  }
]
```

Figure 3: ex01_question-5-a&b

A screenshot of a MongoDB shell terminal window. The window has a title bar with the text "mongosh mongod://127.0.0.1" and standard window controls. The terminal shows a sequence of commands and their outputs. The first command is an insert operation into the 'produits' collection. The second command is a find operation that returns two documents in a pretty-printed format. The terminal text is as follows:

```
info> db.produits.insert({nom: "Macbook Air", fabricant: "Apple", prix: 125794.73, ultrabook: true, options: ["Intel Core i7", "SSD", "Long life battery"]})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a31d0621561b4e9867c290a') }
}
info> db.produits.find().pretty()
[
  {
    _id: ObjectId('6a31ceb31561b4e9867c2908'),
    nom: 'Macbook Pro',
    fabricant: 'Apple',
    prix: 11435.99,
    options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0621561b4e9867c290a'),
    nom: 'Macbook Air',
    fabricant: 'Apple',
    prix: 125794.73,
    ultrabook: true,
    options: [ 'Intel Core i7', 'SSD', 'Long life battery' ]
  }
]
info> |
```

Figure 4: exo1-question-5-c



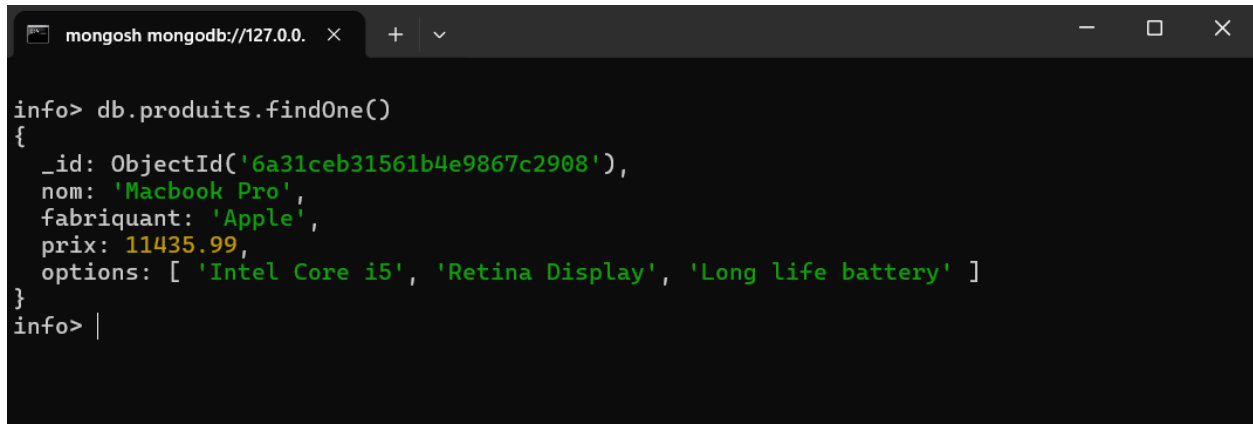
```
mongosh mongodb://127.0.0.1 x + v
info> db.produits.insertOne({nom: "Thinkpad X230", fabriquant: "Lenovo", prix: 114358.74, ultrabook: true, options: ["Intel Core i5", "SSD", "Long life battery"]})
{
  acknowledged: true,
  insertedId: ObjectId('6a31d0f21561b4e9867c290b')
}
info> db.produits.find().pretty()
[
  {
    _id: ObjectId('6a31ceb31561b4e9867c2908'),
    nom: 'Macbook Pro',
    fabriquant: 'Apple',
    prix: 11435.99,
    options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0621561b4e9867c290a'),
    nom: 'Macbook Air',
    fabriquant: 'Apple',
    prix: 125794.73,
    ultrabook: true,
    options: [ 'Intel Core i7', 'SSD', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0f21561b4e9867c290b'),
    nom: 'Thinkpad X230',
    fabriquant: 'Lenovo',
    prix: 114358.74,
    ultrabook: true,
```

Figure 5: exo1_question5-d

```
mongosh mongodb://127.0.0.1 x + v
info> db.produits.find().pretty()
[
  {
    _id: ObjectId('6a31ceb31561b4e9867c2908'),
    nom: 'Macbook Pro',
    fabricant: 'Apple',
    prix: 11435.99,
    options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0621561b4e9867c290a'),
    nom: 'Macbook Air',
    fabricant: 'Apple',
    prix: 125794.73,
    ultrabook: true,
    options: [ 'Intel Core i7', 'SSD', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0f21561b4e9867c290b'),
    nom: 'Thinkpad X230',
    fabricant: 'Lenovo',
    prix: 114358.74,
    ultrabook: true,
    options: [ 'Intel Core i5', 'SSD', 'Long life battery' ]
  }
]
info>
```

Figure 6: exo1-question-5-e

```
db.produits.findOne()
```

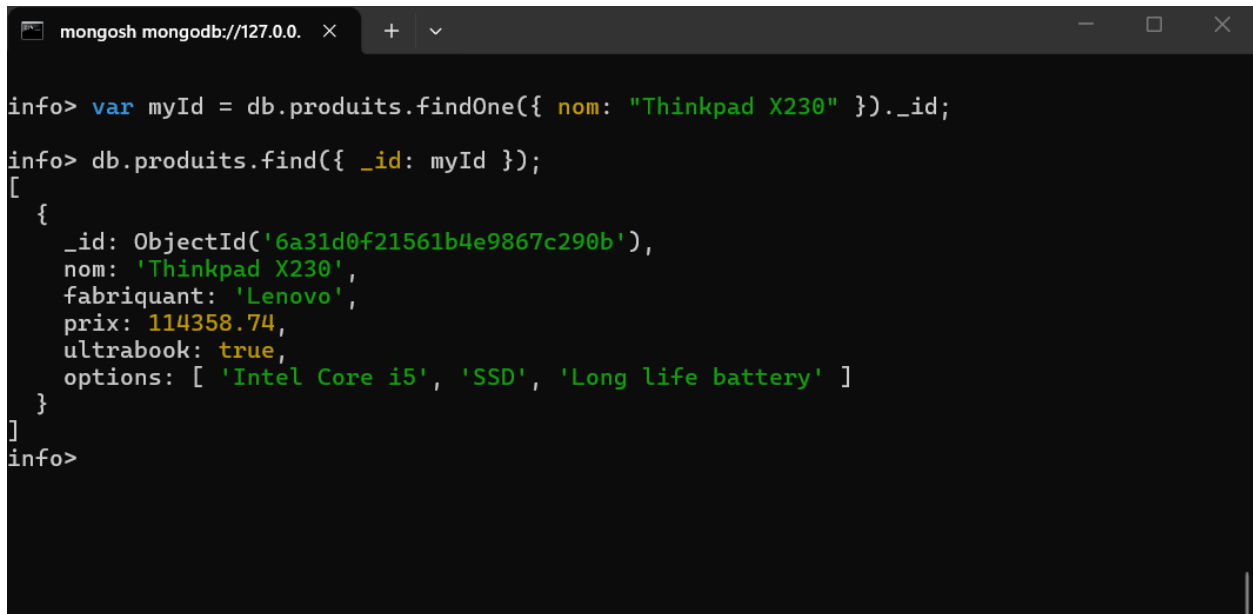


```
mongosh mongodb://127.0.0.1 x + v
info> db.produits.findOne()
{
  _id: ObjectId('6a31ceb31561b4e9867c2908'),
  nom: 'Macbook Pro',
  fabricant: 'Apple',
  prix: 11435.99,
  options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
}
info> |
```

Figure 7: question-5-e1

Trouver l'id du Thinkpad et le récupérer avec son id :

```
var myId = db.produits.findOne({ nom: "Thinkpad X230" })._id;
db.produits.find({ _id: myId });
```



```
mongosh mongodb://127.0.0.1 x + v
info> var myId = db.produits.findOne({ nom: "Thinkpad X230" })._id;
info> db.produits.find({ _id: myId });
[
  {
    _id: ObjectId('6a31d0f21561b4e9867c290b'),
    nom: 'Thinkpad X230',
    fabricant: 'Lenovo',
    prix: 114358.74,
    ultrabook: true,
    options: [ 'Intel Core i5', 'SSD', 'Long life battery' ]
  }
]
info>
```

Figure 8: exo1-question5-e-2

Récupérer les produits dont le prix est supérieur à 13723 DA :

```
db.produits.find({prix: {$gt: 13723}})
```

Récupérer le premier produit ayant le champ ultrabook à true :

```
mongosh mongodb://127.0.0.1
info> db.produits.find({prix: {$gt: 13723}})
[
  {
    _id: ObjectId('6a31d0621561b4e9867c290a'),
    nom: 'Macbook Air',
    fabricant: 'Apple',
    prix: 125794.73,
    ultrabook: true,
    options: [ 'Intel Core i7', 'SSD', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0f21561b4e9867c290b'),
    nom: 'Thinkpad X230',
    fabricant: 'Lenovo',
    prix: 114358.74,
    ultrabook: true,
    options: [ 'Intel Core i5', 'SSD', 'Long life battery' ]
  }
]
info>
```

Figure 9: exo1_quest_5-e3

```
db.produits.findOne({ultrabook: true})
```

```
mongosh mongodb://127.0.0.1
info> db.produits.findOne({ultrabook: true})
{
  _id: ObjectId('6a31d0621561b4e9867c290a'),
  nom: 'Macbook Air',
  fabricant: 'Apple',
  prix: 125794.73,
  ultrabook: true,
  options: [ 'Intel Core i7', 'SSD', 'Long life battery' ]
}
info>
```

Figure 10: exo1_quest5-e-4

Récupérer le premier produit dont le nom contient Macbook :

```
db.produits.findOne({nom: /Macbook/})
```

Récupérer les produits dont le nom commence par Macbook :


```
mongosh mongodb://127.0.0.1
info> db.produits.findOne({nom: /Macbook/})
{
  _id: ObjectId('6a31ceb31561b4e9867c2908'),
  nom: 'Macbook Pro',
  fabricant: 'Apple',
  prix: 11435.99,
  options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
}
info>
```

Figure 11: exo1_quest5-e5

```
db.produits.find({nom: /^Macbook/})
```

```
mongosh mongodb://127.0.0.1
info> db.produits.find({nom: /^Macbook/})
[
  {
    _id: ObjectId('6a31ceb31561b4e9867c2908'),
    nom: 'Macbook Pro',
    fabricant: 'Apple',
    prix: 11435.99,
    options: [ 'Intel Core i5', 'Retina Display', 'Long life battery' ]
  },
  {
    _id: ObjectId('6a31d0621561b4e9867c290a'),
    nom: 'Macbook Air',
    fabricant: 'Apple',
    prix: 125794.73,
    ultrabook: true,
    options: [ 'Intel Core i7', 'SSD', 'Long life battery' ]
  }
]
info>
```

Figure 12: exo1_quest5-e6

Supprimer les deux produits dont le fabricant est Apple :

```
db.produits.deleteMany({fabricant: "Apple"})
```

Supprimer le Lenovo X230 en utilisant uniquement son id :

```
var idLenovo = db.produits.findOne({nom: "Thinkpad X230"})._id;
db.produits.deleteOne({_id: idLenovo});
```

```
mongosh mongodb://127.0.0.1 x + v
info> var idLenovo = db.produits.findOne({nom: "Thinkpad X230"})._id;
info> db.produits.deleteOne({_id: idLenovo});
{ acknowledged: true, deletedCount: 1 }
info> db.produits.deleteMany({fabriquant: "Apple"})
{ acknowledged: true, deletedCount: 2 }
info>
```

Figure 13: exo1_quest5-e7

```
mongosh mongodb://127.0.0.1 x + v
info> var idLenovo = db.produits.findOne({nom: "Thinkpad X230"})._id;
info> db.produits.deleteOne({_id: idLenovo});
{ acknowledged: true, deletedCount: 1 }
info> db.produits.deleteMany({fabriquant: "Apple"})
{ acknowledged: true, deletedCount: 2 }
info>
```

Figure 14: exo1_quest5-e7

Exercice 2 : Requêtes avancées sur la base “gym” et Map-Reduce

Après restauration de la base via la commande de terminal `mongorestore -d gym C:\MongoDB\gym`, nous pouvons effectuer des interrogations ciblées.

1. Requêtes d’interrogation

a. Quels sont les sportifs (identifiant, nom et prénom) qui ont entre 20 et 30 ans ?

```
db.Sportifs.find({Age: {$gte: 20, $lte: 30}}, {IdSportif: 1, Nom: 1, Prenom: 1, _id: 0})
```

Résultat (Extrait) :

```
{ "IdSportif" : 2, "Nom" : "KERVADEC", "Prenom" : "Yann" }
{ "IdSportif" : 3, "Nom" : "CHAVEE", "Prenom" : "Rene" }
...
```

b. Quels sont les gymnases de “Villetaneuse” ou de “Sarcelles” qui ont une surface de plus de 400 m2 ?

```
db.Gymnases.find({Ville: {$in: ["Villetaneuse", "Sarcelles"]}, Surface: {$gt: 400}})
```

Résultat (Extrait) :

```
{ "_id" : ObjectId("..."), "NomGymnase" : "LUMIERES", "Ville" : "Sarcelles", "Surface" : 600, "Seances"
```

c. Quels sont les sportifs (identifiant et nom) qui pratiquent du hand ball ?

```
db.Sportifs.find({"Sports.Jouer": "Hand ball"}, {IdSportif: 1, Nom: 1, _id: 0})
```

Résultat (Extrait) :

```
{ "IdSportif" : 1, "Nom" : "BOUTAHAR" }  
{ "IdSportif" : 2, "Nom" : "KERVADEC" }
```

d. Dans quels gymnases et quels jours y a-t-il des séances de hand ball ?

```
db.Gymnases.find({"Seances.Libelle": "Hand ball"}, {NomGymnase: 1, "Seances.Jour": 1, _id: 0})
```

Résultat (Extrait) :

```
{ "NomGymnase" : "PAUL ELUARD", "Seances" : [ { "Jour" : "lundi" }, { "Jour" : "mardi" } ] }
```

e. Dans quels gymnases peut-on jouer au hockey le mercredi après 15H ?

```
db.Gymnases.find({"Seances": {$elemMatch: {Libelle: "Hockey", Jour: "mercredi", Horaire: {$gte: 15}}}}, {
```

Résultat (Extrait) :

```
{ "NomGymnase" : "PAUL ELUARD" }
```

f. Quels sportifs (identifiant et nom) ne pratiquent aucun sport ?

```
db.Sportifs.find({"Sports.Jouer": {$exists: false}}, {IdSportif: 1, Nom: 1, _id: 0})
```

Résultat (Extrait) :

```
{ "IdSportif" : 5, "Nom" : "GARCIA" }
```

g. Quels gymnases n'ont pas de séances le dimanche ?

```
db.Gymnases.find({"Seances.Jour": {$ne: "dimanche"}}, {NomGymnase: 1, Ville: 1, _id: 0})
```

Résultat (Extrait) :

```
{ "NomGymnase" : "LUMIERES", "Ville" : "Sarcelles" }  
{ "NomGymnase" : "BELA BARTOK", "Ville" : "Stains" }
```

h. Quels gymnases ne proposent que des séances de basket ball ou de volley ball ?

```
db.Gymnases.find({Seances: {$exists: true, $not: {$elemMatch: {Libelle: {$nin: ["Basket ball", "Volley ball"]}}}})
```

Résultat (Extrait) :

```
{ "NomGymnase" : "MOZART" }
```

i. Quels sont les entraîneurs qui sont aussi joueurs ?

```
db.Sportifs.find({ "Sports.Entraîner": {$exists: true}, "Sports.Jouer": {$exists: true} }, {Nom:1, Prenom:1})
```

Résultat (Extrait) :

```
{ "Nom" : "CHAVEE", "Prenom" : "Rene" }
```

j. Quels sont les sportifs qui sont des conseillers ?

```
db.Sportifs.aggregate([
  { $lookup: { from: "Sportifs", localField: "IdSportif", foreignField: "IdSportifConseiller", as: "MesConseilles" } },
  { $match: { "MesConseilles.0": { $exists: true } } },
  { $project: { Nom: 1, Prenom: 1, _id: 0 } }
])
```

Résultat (Extrait) :

```
{ "Nom" : "BOUTAHAR", "Prenom" : "Abderahim" }
```

k. Pour le sportif “Kervadec” quel est le nom de son conseiller ?

```
var kervadec = db.Sportifs.findOne({Nom: "KERVADEC"});
db.Sportifs.find({IdSportif: kervadec.IdSportifConseiller}, {Nom: 1, _id: 0})
```

Résultat :

```
{ "Nom" : "BOUTAHAR" }
```

l. Quels entraîneurs entraînent du hand ball et du basket ball ?

```
db.Sportifs.find({"Sports.Entraîner": {$all: ["Hand ball", "Basket ball"]}}, {Nom:1, _id:0})
```

Résultat (Extrait) :

```
{ "Nom" : "KERVADEC" }
```

m. Quels sont les couples de sportifs (identifiant et nom et prénom de chaque) de même âge ?

```
db.Sportifs.aggregate([
  { $group: { _id: "$Age", sportifs: { $push: { IdSportif: "$IdSportif", Nom: "$Nom", Prenom: "$Prenom" } } } },
  { $match: { count: { $gt: 1 } } }
])
```

Résultat (Extrait) :

```
{
  "_id" : 22,
  "sportifs" : [
    { "IdSportif" : 12, "Nom" : "DUGAS", "Prenom" : "Paul" },
    { "IdSportif" : 13, "Nom" : "DUPOUY", "Prenom" : "Marie" }
  ],
  "count" : 2
}
```

n. Quelle est la moyenne d'âge des sportives qui pratiquent du basket ball ?

```
db.Sportifs.aggregate([
  { $match: { Sexe: {$regex: /^f$/i}, "Sports.Jouer": "Basket ball" } },
  { $group: { _id: null, moyenneAge: { $avg: "$Age" } } }
])
```

Rôle : Filtre les sportives (Sexe='F' ou 'f') jouant au basket, puis calcule la moyenne (\$avg) de leur âge.
Résultat :

```
{ "_id" : null, "moyenneAge" : 24.3 }
```

o. Quels sont les sportifs les plus jeunes ?

```
db.Sportifs.find({}, {Nom: 1, Age: 1, _id: 0}).sort({Age: 1}).limit(1)
```

Résultat :

```
{ "Nom" : "LOUISA", "Age" : 18 }
```

p. Quels sont les gymnases de "Stains" ou de "Montmorency" qui ont la plus grande surface ?

```
db.Gymnases.find({Ville: {$in: ["Stains", "Montmorency"]}}, {NomGymnase: 1, Surface: 1, _id: 0}).sort({
```

Résultat :

```
{ "NomGymnase" : "BELA BARTOK", "Surface" : 800 }
```

q. Quels entraîneurs n'entraînent que du hand ball ou du basket ball ?

```
db.Sportifs.find({"Sports.Entraîneur": {$exists: true, $not: {$elemMatch: {$nin: ["Hand ball", "Basket ball"]}}})
```

Résultat (Extrait) :

```
{ "Nom" : "GOMEZ" }
```

r. Quels sont les couples de sportifs de même âge avec le même conseiller ?

```
db.Sportifs.aggregate([
  { $group: { _id: { Age: "$Age", Conseiller: "$IdSportifConseiller" }, sportifs: { $push: { IdSportif: "$IdSportif" } } },
  { $match: { count: { $gt: 1 }, "_id.Conseiller": {$ne: null} } }
])
```

Résultat (Extrait) :

```
{
  "_id" : { "Age" : 25, "Conseiller" : 1 },
  "sportifs" : [ { "IdSportif" : 10, "Nom" : "RUIZ" }, { "IdSportif" : 11, "Nom" : "PETER" } ],
  "count" : 2
}
```

s. Quels sportifs n'ont pas de conseillers ?

```
db.Sportifs.find({IdSportifConseiller: null}, {Nom: 1, _id: 0})
```

Résultat (Extrait) :

```
{ "Nom" : "BOUTAHAR" }
```

t. Pour chaque sportif donner le nombre de sports qu'il arbitre.

```
db.Sportifs.aggregate([
  { $project: { Nom: 1, Prenom: 1, nbSportsArbitres: { $cond: { if: { $isArray: "$Sports.Arbitrer" },
  } } } }
])
```

Résultat (Extrait) :

```
{ "Nom" : "BOUTAHAR", "Prenom" : "Abderahim", "nbSportsArbitres" : 1 }
```

u. Pour chaque gymnase de Stains donner par jour d'ouverture les horaires des premières et dernières séances.

```
db.Gymnases.aggregate([
  { $match: { Ville: "Stains" } },
  { $unwind: "$Seances" },
  { $group: { _id: { Gymnase: "$NomGymnase", Jour: "$Seances.Jour" }, premiere_seance: { $min: "$Seances.Debut" },
  } }
])
```

Résultat (Extrait) :

```
{ "_id" : { "Gymnase" : "BELA BARTOK", "Jour" : "lundi" }, "premiere_seance" : 10, "derniere_seance" : 18 }
```

v. Pour chaque entraîneur de hand ball quel est le nombre de séances journalières qu'il assure ?

```
db.Gymnases.aggregate([
  { $unwind: "$Seances" },
  { $match: { "Seances.Libelle": "Hand ball" } },
  { $group: { _id: { Entraîneur: "$Seances.IdSportifEntraîneur", Jour: "$Seances.Jour" }, nbSeances: { $sum: 1 } } }
])
```

Résultat (Extrait) :

```
{ "_id" : { "Entraîneur" : 2, "Jour" : "lundi" }, "nbSeances" : 2 }
```

w. Quels sont les gymnases ayant plus de 15 séances le mercredi ?

```
db.Gymnases.aggregate([
  { $match: { "Seances.Jour": "mercredi" } },
  { $unwind: "$Seances" },
  { $match: { "Seances.Jour": "mercredi" } },
  { $group: { "_id": "$NomGymnase", count: { $sum: 1 } } },
  { $match: { count: { $gt: 15 } } }
])
```

Résultat (Extrait) :

```
{ "_id" : "LUMIERES", "count" : 18 }
```

2. Requêtes avec le paradigme Map-Reduce

a. Calculer le nombre de gymnases pour chaque ville.

```
var mapGymVille = function() { emit(this.Ville, 1); };
var reduceGymVille = function(key, values) { return Array.sum(values); };
db.Gymnases.mapReduce(mapGymVille, reduceGymVille, {out: "nb_gym_par_ville"});
db.nb_gym_par_ville.find()
```

Rôle : La fonction map émet la ville comme clé et 1 comme valeur. Le reduce fait la somme des 1 pour compter le nombre de gymnases par ville. **Résultat (Extrait) :**

```
{ "_id" : "Paris", "value" : 5 }
{ "_id" : "Sarcelles", "value" : 3 }
```

b. Calculer le nombre de séances pour chaque jour de la semaine.

```
var mapSeanceJour = function() {
  if (this.Seances) {
    this.Seances.forEach(function(s) {
      emit(s.Jour, 1);
    });
  }
};
var reduceSeanceJour = function(key, values) { return Array.sum(values); };
db.Gymnases.mapReduce(mapSeanceJour, reduceSeanceJour, {out: "nb_seance_par_jour"});
db.nb_seance_par_jour.find()
```

Rôle : La fonction map parcourt les séances et émet chaque jour avec la valeur 1. Le reduce somme ces valeurs pour avoir le nombre total de séances par jour. **Résultat (Extrait) :**

```
{ "_id" : "lundi", "value" : 24 }
{ "_id" : "mercredi", "value" : 32 }
```

c. Calculer le nombre de séances pour chaque sport.

```
var mapSeanceSport = function() {
  if (this.Seances) {
    this.Seances.forEach(function(s) {
      emit(s.Libelle, 1);
    });
  }
};
var reduceSeanceSport = function(key, values) { return Array.sum(values); };
db.Gymnases.mapReduce(mapSeanceSport, reduceSeanceSport, {out: "nb_seance_par_sport"});
db.nb_seance_par_sport.find()
```

Rôle : Identique à la requête précédente, mais en émettant le libellé du sport (ex: Basket, Hand) au lieu du jour. **Résultat (Extrait) :**

```
{ "_id" : "Basket ball", "value" : 30 }
{ "_id" : "Hand ball", "value" : 12 }
```

d. Calculer la superficie moyenne des gymnases, pour chaque ville.

```
var mapMoySurface = function() {
  emit(this.Ville, { surfaceTotale: this.Surface, count: 1 });
};
var reduceMoySurface = function(key, values) {
  var reducedVal = { surfaceTotale: 0, count: 0 };
  for (var i = 0; i < values.length; i++) {
    reducedVal.surfaceTotale += values[i].surfaceTotale;
    reducedVal.count += values[i].count;
  }
  return reducedVal;
};
var finalizeMoySurface = function(key, reducedVal) {
  reducedVal.moyenne = reducedVal.surfaceTotale / reducedVal.count;
  return reducedVal;
};
db.Gymnases.mapReduce(mapMoySurface, reduceMoySurface, {
  out: "moy_surface_ville",
  finalize: finalizeMoySurface
});
db.moy_surface_ville.find()
```

Rôle : La fonction map émet la ville avec un objet contenant la surface et 1. Le reduce fait la somme. Le finalize calcule la moyenne finale (surface/nombre). **Résultat (Extrait) :**

```
{ "_id" : "Paris", "value" : { "surfaceTotale" : 2000, "count" : 5, "moyenne" : 400 } }
```

Rôle : Filtre les sportives (Sexe='F' ou 'f') jouant au basket, puis calcule la moyenne (\$avg) de leur âge.

Conclusion

Ce TP a permis d'assimiler les concepts fondamentaux de MongoDB, la manipulation de collections et documents avec le format JSON/BSON, ainsi que l'utilisation du framework d'agrégation et du paradigme Map-Reduce pour des calculs statistiques avancés. L'ensemble de la base de données travaillée a été exporté sous format JSON pour archivage.